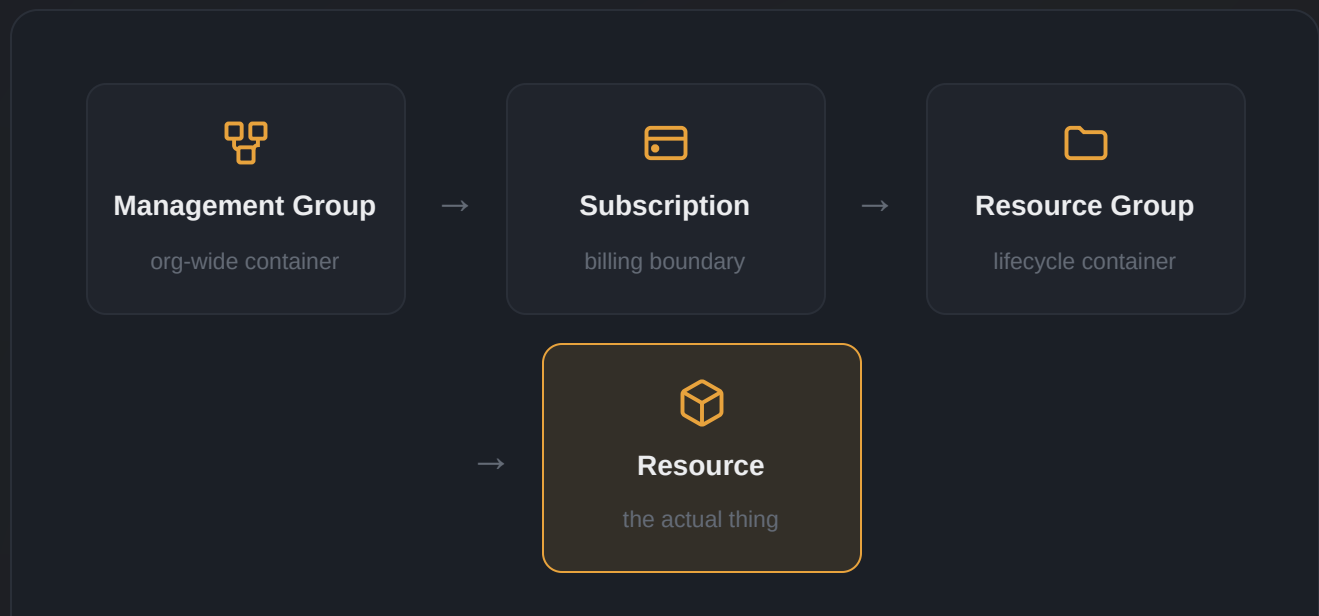


Manage Subscriptions & Governance

How Azure environments are organized and controlled at scale: management groups, subscriptions, Azure Policy, resource locks, tags, and cost management — what each one does, and which tool to reach for.

Why governance exists

RBAC controls *who* can act. Governance controls *what's allowed to exist* and *how it's organized* — consistent configuration, cost visibility, and protection from accidental change, applied consistently across every subscription in an organization rather than resource by resource.



Policy, RBAC, and Locks all flow downward through this chain – apply once at the top, and every level beneath inherits it.

Eight terms you need cold

The vocabulary behind almost every governance question on the exam.

Management Group

A container above subscriptions used to organize several subscriptions under shared policy and RBAC. Supports up to 6 levels deep beneath the root.

Tip: Apply a policy once at the management group instead of repeating it per subscription.

Subscription

Both a billing boundary and a deployment scope. Resource quotas and limits are tracked per subscription.

Tip: Use separate subscriptions to isolate billing or hard limits between environments — e.g. prod vs. dev.

Resource Group

A logical container for resources sharing the same lifecycle. Deleting the group deletes everything inside it.

Tip: A resource belongs to exactly one resource group, even if the group itself spans multiple regions.

Azure Policy

A rule that evaluates resources against a condition and enforces an effect — deny, audit, modify, etc. — on every create or update.

Tip: Policy governs configuration, not identity. It pairs with RBAC; it never replaces it.

Initiative

A bundle of multiple policy definitions assigned and tracked together as one set, often mapped to a compliance standard (NIST, ISO 27001, CIS).

Tip: Roll out a whole compliance framework as one initiative instead of dozens of separate policies.

Policy Effect

What happens when a policy fires: Deny, Audit, Append, Modify, DeployIfNotExists, AuditIfNotExists, or Disabled. See the table below.

Resource Lock

A setting that blocks delete (CanNotDelete) or all write/delete (ReadOnly) on a resource, regardless of the caller's RBAC role.

Tip: Locks inherit downward through scope, exactly like RBAC and Policy.

Tag

A name/value pair attached to a resource for organization, automation targeting, or cost allocation.

Tip: Tags don't cascade to existing resources automatically — enforce them with a Policy Modify effect plus a remediation task.

Policy effects, side by side

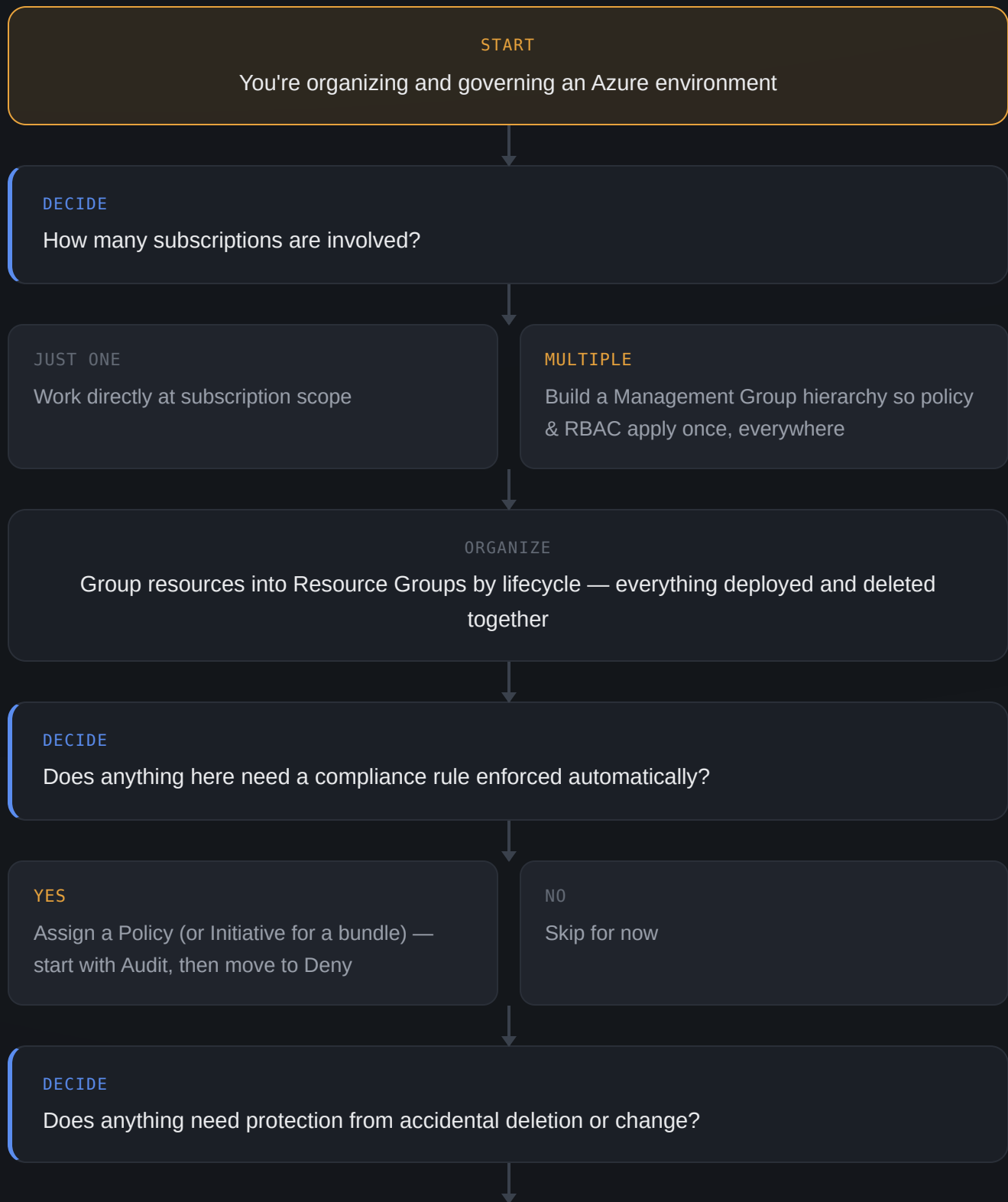
EFFECT	WHAT IT DOES
Deny	Blocks the request outright if it violates the policy
Audit	Allows the request through, but flags the resource as non-compliant
Append	Adds extra fields to the request before the resource is created
Modify	Adds, updates, or removes properties/tags on resources — requires a managed identity
DeployIfNotExists	Triggers a deployment of a related resource when a condition isn't met
AuditIfNotExists	Flags non-compliance when a related resource is missing
Disabled	Turns the effect off entirely — useful while testing a definition

EXAM GOTCHA

Always roll out a new policy as **Audit** first and review the compliance report before switching it to **Deny**. Deny blocks deployments the moment it's assigned — including your own next deployment, if it doesn't comply.

Setting up governance for an environment

Run through this when you're standing up a new subscription or organizing an existing one.



DELETE ONLY

CanNotDelete lock

DELETE & MODIFY

ReadOnly lock

NEITHER

Skip

TAG

Apply tags for owner, environment, and cost-center tracking

DECIDE

Do you need to track or control spend?

YES

Set a Budget with alert thresholds, review
Azure Advisor cost recommendations

NO

Skip

VERIFY

Check the Policy compliance dashboard and the Cost Management overview

DONE

Environment is organized, compliant, and cost-visible

Five ways to apply governance

Management groups, policy assignments, locks, tags, and budgets can all be set through any of these — pick based on whether it's exploratory, scripted, or part of a repeatable deployment.

Azure Portal

BEST FOR: exploring built-ins, compliance dashboards

Browse hundreds of built-in policy definitions before writing a custom one, and see compliance state and cost visually in one place.

```
Management Groups → Policy → Locks  
→ Tags → Cost Management +  
Billing
```

Azure CLI

BEST FOR: scripting, CI/CD pipelines

Cross-platform and easy to chain — create a management group, assign a policy, and lock a resource group in one script.

```
az account management-group create  
--name "mg-prod"  
  
az policy assignment create \  
  --name "require-tag" \  
  --policy "<policy-definition-id>" \  
  --scope "/subscriptions/<sub-id>"  
  
az lock create --name "no-delete" \  
  --lock-type CanNotDelete \  
  --resource-group "rg-name"
```

> Azure PowerShell

BEST FOR: Windows-centric / enterprise scripting

Fits naturally next to the rest of the Az PowerShell module for teams already standardized on it.

```
New-AzManagementGroup -GroupId "mg-prod"
```

```
New-AzPolicyAssignment -Name "require-tag" `
  -PolicyDefinition $def `
  -Scope "/subscriptions/<sub-id>"
```

```
New-AzResourceLock -LockName "no-delete" `
  -LockLevel CanNotDelete `
  -ResourceGroupName "rg-name"
```

<> Bicep / ARM

BEST FOR: infrastructure as code, GitOps

Policy assignments and locks deploy as part of the same template as the resources they govern — version-controlled and peer-reviewed in Git.

```
resource lock
'Microsoft.Authorization/locks@2020-05-01' = {
  name: 'no-delete'
  properties: { level:
    'CanNotDelete' }
}
```

```
resource policyAssignment

'Microsoft.Authorization/policyAssignments@2022-06-01' = {
  name: 'require-tag'
  properties: { policyDefinitionId:
    policyDefId }
}
```

🌐 REST API

BEST FOR: custom tooling

Every tool above calls this underneath — reach for it directly when building integrations outside Microsoft's standard tooling.

```
PUT
https://management.azure.com/{scope}/
providers/Microsoft.Authorization/
policyAssignments/{name}
?api-version=2022-06-01
```

TOOL	BEST FOR	REPEATABLE	LEARNING CURVE
Portal	Exploring & compliance dashboards	No	Low
CLI	Scripting / CI-CD	Yes	Medium
PowerShell	Windows / enterprise scripting	Yes	Medium
Bicep / ARM	IaC, GitOps	Yes	Medium–High
REST API	Custom integrations	Yes	High

Principles worth internalizing

Govern at the highest scope that makes sense, test before you enforce, and remember locks and budgets don't behave the way their names suggest.

LOCK TYPE	BLOCKS	STILL ALLOWS
CanNotDelete	Delete	Read and modify
ReadOnly	Delete, modify, and any action that needs an internal write call	Pure read operations only

EXAM GOTCHA

A **ReadOnly** lock can break things you wouldn't expect: restarting a VM, rotating a Key Vault secret, or scaling a resource all require an internal write call and will fail — even though you're not changing the resource's definition.

COST MANAGEMENT REALITY CHECK

Budgets only alert — crossing a threshold triggers a notification, it doesn't stop spending. If you need a hard stop, pair the budget's action group with an Automation runbook or Logic App that actually deallocates or shuts down resources.

Six mistakes that show up on the exam

Going straight to Deny

New policies should run as Audit first — Deny blocks deployments immediately, including ones you intended to allow.

Locking a resource group, then being surprised

A CanNotDelete or ReadOnly lock blocks everyone, including Owners, until the lock is explicitly removed.

Expecting tags to backfill

A tagging policy only applies to new or updated resources — existing ones need a remediation task to catch up.

Treating a budget as a spending cap

Budgets fire alerts, not brakes — nothing stops being billed unless you wire up automation yourself.

Assigning the same policy to every subscription

If multiple subscriptions need the same rule, assign it once at a shared management group instead.

Building too deep a hierarchy

Management groups support up to 6 levels beneath the root — a flat, well-planned structure is easier to govern than a deep one.